BuyWise Overview

Last updated: May 8, 2026

This document explains the current version of BuyWise in enough detail for a person or GPT to understand what the website does, what is implemented, what is mocked, what is planned but not built, and how the main flows work.

BuyWise is a Next.js, TypeScript, and Tailwind CSS MVP for helping regular people decide whether a product link is worth buying from. The current product direction is link-first: users should be able to drop a resale listing or retail product page and get a verdict on whether that place is a good buy.

The current core promise is:

Drop a product link. BuyWise tells you if it is the best place to buy.

The app is built around protecting buyers before they message, meet a seller, or check out from a retailer. It compares the link price against mock used fair-market data and retail MSRP benchmarks, looks for risky seller wording, checks for trust signals, gives an offer or buy-under range, suggests seller questions, and shows better resale or retail alternatives when the current mock catalog has them.

Current Product Positioning

BuyWise is not meant to be only a price guide. Price guides answer what an item might be worth. BuyWise is meant to answer whether this specific link is worth buying from.

The main user is a normal buyer browsing Facebook Marketplace, Craigslist, OfferUp, or eBay. The first target users include:

- Students buying used laptops or MacBooks.
- Creators buying used cameras.
- People buying bikes, monitors, electronics, or tools.
- Parents buying used gear for kids.
- Anyone nervous about overpaying or getting scammed.

Professional resellers and power buyers are not the first target user. They may become a paid/pro user segment later, but the MVP is built for beginner-friendly, practical buyer protection.

Current product priorities are:

1. Avoid scams.
2. Give a fast yes/no style verdict.
3. Help users save money.
4. Tell users what questions to ask.
5. Help users negotiate better.

The verdict style is strict and cautious, but plain and practical. A cheap resale listing with sketchy wording should not be treated as a great deal. A retail sale should still be compared against used prices and MSRP. In BuyWise, price alone is not enough.

Current Tech Stack

BuyWise currently uses:

- Next.js App Router.
- TypeScript.
- React client and server components.
- Tailwind CSS.
- Supabase client, auth, and SQL schema scaffolding.
- Recharts for depreciation charts.
- lucide-react for icons.
- Mock marketplace services structured so real providers can be added later.
- Browser local storage/session storage for local fallback behavior.

Important project folder:

`/Users/lukeshewmaker/Documents/Codex Sandbox/BuyWise`

Local preview:

`http://localhost:3000/`

Main local commands:

```
npm run dev
npm run typecheck
npm run lint
```

Known local development issue:

If the page ever looks like raw HTML or has no styling, it is usually because `.next` became stale after running `npm run build` while the dev server was open. The known fix is:

1. Stop the dev server.
2. Delete `.next`.
3. Restart `npm run dev`.

Current Route Map

The app currently has these user-facing routes:

- `/` - Homepage focused on dropping a resale or retail product link.
- `/submit` - Full analyzer page reached after the homepage link prompt or from product pages.
- `/search` - Product search and price guide browsing.
- `/products/[id]` - Product insight pages for individual mock products.
- `/saved` - Saved items list and status tracking.
- `/auth` - Supabase login/signup page.
- `/admin` - Mock data/admin overview page.
- `/about` - Customer-facing page explaining what BuyWise does and why buyers should use it.
- `/monetization` - Redirects to `/about` so old links do not break.
- `/api/extract-link` - Server-side helper used by the homepage and analyzer to read safe product/listing page metadata when possible.
- Not-found page - Shown when a product id does not exist.

The navigation currently includes:

- BuyWise logo.
- Centered links for Search, Analyze Listing, and Saved.
- Right-side About Us text link styled like the main navigation links.
- Right-aligned black Log In button linking to `/auth`, the same account page as the footer Account link.

The BuyWise logo and Analyze Listing nav item both link to `/`, because the homepage is the main pasted-link analyzer experience.

The footer includes:

- A note that BuyWise uses mock market data for the MVP.
- Links to Search, Analyze Listing, Saved, About Us, and Account.

Homepage

The homepage is now built around the main product action: drop a resale or retail product link and analyze it.

The hero headline is:

Drop a product link. Know if it is the best place to buy.

The homepage explains that BuyWise rates the bargain, checks risk, and compares the link against used and retail alternatives when the current catalog has a better option.

The homepage emphasizes four core capabilities:

- Rate resale listings.
- Rate retail bargains.
- Compare used vs new.
- Show better alternatives when the catalog has them.

The homepage's main call to action is the link checker panel. It asks for:

- Listing or product link.
- Link type: resale listing or retail bargain.
- Optional extra details only if the user has them.

The quick homepage prompt now attempts to auto-populate from the pasted link instead of asking for product/model and price up front. It calls `/api/extract-link` to pull the page title, price, source, rough description, link mode, and closest mock-catalog match when the page exposes readable metadata. If the site blocks access or does not expose a reliable price/model, the full analyzer asks for the missing details on `/submit`.

The homepage still stores a draft in browser session storage and sends the user to `/submit`, where the user can review or correct the auto-filled details and run the full analyzer.

The homepage prompt can infer some source/type details from the URL. For example, eBay and Craigslist links are treated as resale by default, while Best Buy, Amazon, Walmart, Target, Apple, Dell, B&H Photo, Adorama, Sony, Canon, Trek, and similar retailer/manufacturer links are treated as retail by default.

The storage key is:

```
buywise.listingDraft.v1
```

If the user fills any draft field and submits the homepage form, the draft is saved and loaded into the full analyzer on `/submit`. If the user submits an empty homepage form, no draft is stored.

The homepage also includes:

- A "What you get back" section that explains the result in buyer language: verdict, money, proof, risk check, price check, better options, and next move.
- An "Example Products" section with four evenly spaced product cards: Sony A6400, MacBook Air M1, Trek FX 3, and Dell UltraSharp U2720Q.
- A right-aligned Browse all products link in that product section.
- A condensed example analysis section below the product cards. It shows a "Risky purchase" verdict, listing context, deal score, scam probability, fair used price, suggested offer, reasons the listing may not be worth it, and seller questions to ask next.

The homepage intentionally feels like a practical buyer tool, not a generic SaaS landing page.

Full Analyzer Page

The `/submit` page is the full analyzer step behind the homepage flow. It is no longer a main navbar destination because the homepage is the primary Analyze Listing surface.

Its purpose is to let a user review auto-filled link details, add anything missing, and get a practical buyer verdict.

The page tells the user to paste:

- Listing or product link.
- Link price.
- Product or model.
- Link type: resale listing or retail bargain.
- Description or page details.
- Marketplace/source.
- Seller replies or extra deal notes.
- Condition or deal state.
- Location if available.

The page returns:

- A verdict.
- Red flags.
- Suggested offer range for resale links or target buy range for retail links.
- Better resale alternatives when the mock catalog has them.
- Better retail/MSRP alternatives when the mock catalog has them.
- Questions to ask.
- Meetup checklist.
- Confidence score.
- Next steps before messaging, meeting, or checking out.

The top of the page gives three usage tips:

- Start with the link.
- Compare both sides.
- Act only when better.

These reinforce the main product philosophy: a resale link may lose to buying new, and a retail sale may lose to used pricing.

Listing Analyzer Form

The main form lives in:

`components/ListingAnalyzerForm.tsx`

The analyzer is a client component for form state and result display. It still uses local/mock deal scoring, but it now calls the server-side `/api/extract-link` route to auto-populate fields from safe public links. There is no OpenAI API route yet and no paid API usage.

The analyzer supports these fields:

- Listing or product link.
- Link type: resale listing or retail bargain.
- Product or model.
- Closest price guide.
- Link price.
- Condition or deal state.
- Marketplace/source.
- Location.
- Listing text or retail page details.
- Seller replies or extra deal notes.

When the user pastes a link, the analyzer waits briefly and then tries to read the page. If successful, it can fill:

- Link type.
- Marketplace/source.
- Product or model.
- Link price.
- Listing text or retail page details.
- Closest matched mock price guide.

The user can edit anything that was filled automatically. Auto-fill is treated as a starting point, not a final source of truth.

The supported marketplace values are:

- Facebook Marketplace.
- Craigslist.
- OfferUp.
- eBay.
- Local seller.
- Other.

Supported condition values are:

- Like new.

- Excellent.
- Good.
- Fair.
- Poor.
- For parts or repair.

The "Closest price guide" field points to the mock product catalog. The current MVP only has mock price guides for cameras, laptops, bikes, and monitors.

The analyzer tries to find the closest product by matching the entered product name against each mock product's brand and model. If it cannot find a match, it falls back to the currently selected mock product in the dropdown.

Link price is required. It must be a positive number. If the user submits an invalid price, the form shows an error:

Enter a valid price before analyzing.

The form has a loading state while analysis runs. The button changes to "Checking link" with a spinner. The current analysis is fast because it is local logic, but the state is already structured for future async behavior.

Before a result is generated, the right side of the analyzer shows an empty state that explains what to paste:

- For resale links: seller wording, condition, proof, and pickup notes.
- For retail links: product name, sale price, retailer, warranty, and deal notes.
- BuyWise compares the link against used fair value and retail MSRP benchmarks.

The empty state also shows a buyer rule:

A low resale price still needs proof. A retail sale still needs comparison against used prices and MSRP.

Link Analysis Helpers

Link analysis helpers live in:

`lib/linkAnalysis.ts`

This file supports the link-first direction by keeping URL/source inference and mock alternative building separate from the server-side extraction route.

It currently does four jobs:

- Infer source labels from URL hostnames.
- Infer whether a URL is resale or retail.
- Infer marketplace source for supported resale domains.
- Build mock resale and retail/MSRP alternatives from the existing product catalog.

Current resale-style domains include:

- eBay.
- Craigslist.
- Facebook.
- OfferUp.
- Mercari.
- Swappa.
- Depop.
- Poshmark.

Current retail-style domains include:

- Amazon.
- Best Buy.
- Walmart.
- Target.
- Costco.
- B&H Photo.
- Adorama.
- Apple.
- Dell.
- Lenovo.
- Canon.
- Sony.
- Trek.
- Specialized.

The app now has basic server-side page metadata extraction for safe, public links. It tries to fetch a readable HTML page and extract title, description, structured product price, JSON-LD Product/Offer data, Open Graph/Twitter metadata, and a fallback visible dollar price. It also tries to match the extracted title/description against the mock product catalog.

This is not full marketplace integration. BuyWise does not scrape Facebook Marketplace, does not read listing images, does not inspect seller profiles, and does not guarantee every retailer or marketplace can be read. Many sites block automated page reads or hide prices in client-side scripts. When that happens, BuyWise falls back to manual entry.

The URL is still used for source/type detection and context even when page reading fails. The user can always paste price/model/details manually for the local analyzer.

Link Extraction API

The link extraction route lives in:

`app/api/extract-link/route.ts`

It is called by:

- `components/HomeListingPrompt.tsx`
- `components/ListingAnalyzerForm.tsx`

The route accepts a pasted URL and returns a `LinkExtractionResult`.

It can return:

- Normalized URL.
- Source label.
- Source domain.
- Inferred mode: resale or retail.
- Marketplace source when recognized.
- Page title.
- Page description.
- Extracted price.
- Matched mock-catalog product name.
- Confidence score.
- Human-readable message.
- Warnings when price or product match is missing.
- Manual fallback flag.

The route includes safety checks:

- Only `http` and `https` URLs are accepted.
- Localhost and private/internal network addresses are blocked.
- DNS results are checked for private/local addresses before fetching.
- Fetching has a timeout.
- HTML reads are limited in size.
- Facebook Marketplace and Facebook links are not fetched; the user is told to paste details manually.

Extraction sources currently include:

- Open Graph title and description.
- Twitter card title and description.
- Standard meta description.

- Product price meta fields.
- JSON-LD `Product` and `Offer` data.
- Page `<title>`.
- Fallback dollar-price text found in the readable HTML.

This route is intentionally conservative. If it cannot confidently extract useful details, it does not invent them. It returns a manual-entry message instead.

The helper builds a `ListingAnalysisContext` with:

- Mode: resale or retail.
- Listing URL.
- Source label.
- Source domain.
- Link price.
- Benchmark label.
- Matched product name.
- Resale alternatives.
- Retail/MSRP alternatives.

Resale alternatives are selected from the same mock category when the fair used price is meaningfully below the pasted link price. The selected product itself can also appear as a resale alternative when its fair used price is much lower than the link price. Each alternative now includes a buyer action and expected outcome, not just a price comparison.

Retail/MSRP alternatives are selected from the same mock category when MSRP is lower than the pasted link price. For resale links, the selected product's MSRP can appear as a retail benchmark when new MSRP is close enough to the used link price that buying new may be worth comparing. Each retail alternative also includes a recommended action and practical outcome.

Analyzer Result UI

The analyzer result card lives in:

`components/DealScoreCard.tsx`

The result is designed as a buyer decision panel, not only a scorecard.

The top of the result shows:

- Resale listing verdict or retail bargain verdict.
- Recommendation badge.
- Save verdict button when there is a matched product and link context.
- Source label, matched product, and link price when link context exists.

- Plain-English explanation.
- Suggested offer range for resale links or target buy range for retail links.
- Negotiation tip or buy-under guidance.

The result includes four summary metrics:

- Deal score, from 0 to 100.
- Risk level: Low, Medium, or High.
- Confidence score, from 0 to 100.
- Benchmark gap, shown as dollars over or under the relevant benchmark.

The result then shows:

- Why this might NOT be worth it.
- Red flags.
- Trust signals.
- Better resale moves.
- Better retail moves.
- Next steps before buying.
- Questions to ask.
- Meetup checklist or before-checkout checklist.

The "Why this might NOT be worth it" section is always visible. It is meant to build trust by clearly explaining the downside before pushing the user toward action. It can include:

- Red flags found in the pasted text.
- Price being above the relevant benchmark.
- Medium or high risk level.
- Low confidence due to missing proof.
- Lack of trust signals.
- Retail being close enough to avoid used-item risk.
- Used alternatives beating a retail deal.

If there is no major blocker from the current details, the section says that directly while still reminding the user to verify condition, source, and payment/return safety.

The "Questions to send" panel includes a Copy button. It copies seller questions to the clipboard. When copied, the button temporarily changes to "Copied."

The "Save verdict" button bookmarks the analyzed link into the Saved tab. It stores the matched product, analyzed link price, source, seller location if available, verdict summary, deal score, confidence score, source label, original link when present, and the main concern if one exists. If Supabase is configured and the user is logged in, it writes to `saved_items`; otherwise it uses browser local storage.

If a matched product exists, the result uses that product's category-specific seller questions and buying checklist. If no product is available, it falls back to generic safety questions and checklist items.

Fallback questions for high risk include:

- Can you send a fresh working video with today's date in the shot?
- Do you have the receipt or serial number?
- Can I inspect it in person before any payment?

Generic fallback questions include:

- Can you send a short video showing it working?
- Are there any issues not shown in the listing?
- Can we meet somewhere public where I can test it?

Generic fallback checklist items include:

- Verify ownership before meeting.
- Test the item before paying.
- Meet publicly.
- Do not send money before inspection.

The result card uses different subtle background/border styling based on the recommendation:

- Great deal.
- Fair price.
- Overpriced.
- Risky purchase.
- Avoid.

Risk level also has its own visual treatment:

- Low: green.
- Medium: amber.
- High: red.

If link context exists, the result card shows two alternatives panels:

- Better resale moves: used-side replacement actions if the pasted link is overpriced, risky, or not clearly better than the market.
- Better retail moves: new-product fallbacks to check when warranty, returns, or a close retail price may beat the pasted link.

Each alternative card shows:

- Alternative product title.
- Price label.
- Price.
- Reason the alternative is shown.
- Recommended action.
- Expected buyer outcome.
- Link to the relevant product price guide.

If no better alternative exists in the current mock catalog, the relevant panel says so directly.

Local Deal Scoring Logic

The core analyzer logic lives in:

`lib/dealQuality.ts`

The main exported function is:

`calculateDealQuality(input)`

The input includes:

- Asking price.
- Fair price or MSRP benchmark, depending on link mode.
- Used low.
- Used high.
- Reliability score.
- Scam probability score.
- Condition.
- Marketplace source.
- Listing text.

The output includes:

- Deal score.
- Recommendation label.
- Explanation.
- Suggested offer low.
- Suggested offer high.
- Risk level.

- Price difference.
- Confidence score.
- Red flags.
- Positive signals.
- Negotiation tip.
- Next steps.

Supported recommendation labels are:

- Great deal.
- Fair price.
- Overpriced.
- Risky purchase.
- Avoid.

Supported risk levels are:

- Low.
- Medium.
- High.

Red Flag Detection

The analyzer checks the condition text and listing text for red flag wording.

Current red flag categories:

- Payment before pickup.
- Urgency pressure.
- Stock photos only.
- No proof of ownership.
- Refuses safe meetup.
- Lock or ownership warning.
- Broken or parts listing.
- Price far below market.

Examples of risky terms include:

- pay before.
- payment before.
- deposit.

- zelle first.
- zelle deposit.
- cashapp first.
- cash app first.
- venmo first.
- venmo deposit.
- wire.
- western union.
- must go today.
- need gone today.
- first come first serve.
- no holds.
- moving today.
- urgent.
- priced to sell today.
- stock photo.
- stock photos.
- photos from website.
- google image.
- not actual photo.
- no receipt.
- lost receipt.
- no serial.
- serial removed.
- cannot provide serial.
- ship only.
- no meetup.
- cannot meet.
- will not meet.
- payment only.
- delivery only.
- iCloud locked.
- activation lock.
- MDM.
- BIOS password.

- managed device.
- locked account.
- for parts.
- as-is.
- broken.
- does not turn on.
- cracked.
- water damage.

The "Price far below market" red flag is added when the asking price is 65 percent or less of fair price. This reflects the product belief that a price far below market should require extra proof.

Positive Signal Detection

The analyzer checks for trust signals in listing text.

Current positive signal categories:

- Receipt or proof of purchase.
- Serial number available.
- Original box included.
- Working video available.
- Can test before buying.
- Public meetup.
- Warranty mentioned.
- Battery health listed.
- Shutter count listed.

Examples of positive terms include:

- receipt.
- original receipt.
- receipt included.
- proof of purchase.
- serial number.
- serial available.
- original box.
- video available.
- working video.

- video of it working.
- can send video.
- can test.
- test before buying.
- public meetup.
- meet in public.
- meet at police station.
- meet at library.
- local pickup.
- warranty.
- battery health.
- shutter count.

Positive signals can improve confidence and slightly improve the score, but they do not fully cancel serious red flags.

Marketplace Risk Adjustment

The analyzer adjusts effective scam probability based on marketplace:

- eBay: -1.
- Craigslist: +1.
- Facebook Marketplace: +2.
- OfferUp: +2.
- Local seller: +1.
- Other: +1.

This does not mean one marketplace is always unsafe or safe. It is a simple MVP risk adjustment based on the fact that local marketplaces often require more buyer verification.

Condition Adjustment

Poor condition terms reduce the deal score:

- poor.
- for parts.
- broken.
- repair.
- damaged.
- cracked.
- activation lock.

- iCloud.
- locked.
- MDM.
- stolen.

Good condition terms can slightly improve the score:

- excellent.
- mint.
- like new.
- open box.
- new.

The condition value "Fair" also applies a negative adjustment.

Recommendation Logic

Recommendation is based on:

- Asking price compared with fair price.
- Effective scam probability.
- Reliability score.
- Red flags.

Important behavior:

- If reliability is very low and price is high, recommendation becomes Avoid.
- If high risk signals exist, recommendation becomes Risky purchase.
- If the item is 20 percent or more below fair price and effective scam probability is low enough, it can become Great deal.
- If price is at least 15 percent above fair price, it can become Overpriced.
- If price is roughly within 10 percent of fair value, it can become Fair price.
- If price looks low but risk is medium/high, it can become Risky purchase.

This is why a very cheap listing may not receive "Great deal." Risk can override price.

Deal Score

The deal score is calculated from:

- Price score.
- Reliability adjustment.

- Scam probability adjustment.
- Condition adjustment.
- Red flag adjustment.
- Positive signal adjustment.

The final score is clamped between 0 and 100.

The price score favors lower prices but is not the only factor. Serious risk can drag down a low-priced listing.

Risk Level

Risk level is High when:

- Scam probability is high.
- Reliability is very low.
- Condition text has poor/broken/locked terms.
- Any high-severity red flag exists.

Risk level is Medium when:

- Scam probability is moderate.
- Reliability is moderate/low.
- Any red flags are present.

Otherwise, risk level is Low.

Confidence Score

Confidence score is based on:

- Length/detail of listing text.
- Whether asking price is inside the used low/high range.
- Number of positive proof signals.
- Red flag penalties.

The score is clamped from 30 to 96.

Longer listings with real proof signals produce higher confidence. Sparse listings with no seller detail produce lower confidence.

Suggested Offer Range

The offer range is based on:

- Asking/link price.
- Fair price.
- Used low.
- Used high.
- Condition risk.

If the asking price is below fair value, the suggested offer range is usually near the asking price, roughly 92 percent to 98 percent of asking, bounded by used low and fair price.

If the asking price is above fair value, the suggested offer range is based on fair value, usually around 85 percent to 95 percent of fair value, bounded by used low and used high.

If condition is poor, the base offer range is lower.

Negotiation Tip

Negotiation tip changes based on risk and price:

- High risk: Do not negotiate until the seller proves ownership and shows the item working.
- Overpriced: Open below fair value and mention recent sold prices, not asking prices.
- Underpriced: Ask verification questions first, then move quickly if it checks out.
- Fair price: Offer slightly under fair value and leave room to meet in the middle.

Next Steps

Next steps change based on risk:

High risk next steps:

- Ask for a fresh working video with today's date or your name in the shot.
- Get proof of ownership, receipt, or serial number before meeting.
- Meet publicly and do not send payment before inspection.

Listings with red flags:

- Clarify every red flag before making an offer.
- Ask category-specific inspection questions.
- Use the suggested offer range only after the seller answers clearly.

Overpriced listings:

- Send a fair offer with one sentence explaining the market range.
- Ask for proof of condition before driving there.
- Be ready to pass if the seller will not move on price.

Lower-risk listings:

- Ask for proof it works and verify ownership.
- Schedule a public meetup where you can inspect it.
- Use the checklist before handing over payment.

Product Search

The search page is available at:

`/search`

Search is secondary to the listing analyzer, but it still matters as a reference tool.

The search page lets users:

- Search by product name.
- Search by category.
- Search by brand.
- Search by model.
- Search by year.
- Search by common issue text.
- Filter by category.
- Filter by minimum year.

Supported categories are:

- Cameras.
- Laptops.
- Bikes.
- Monitors.

The search logic lives in:

`lib/search.ts`

Search checks a combined lowercased text string containing:

- Category.
- Brand.
- Model.
- Year.

- Brand plus model.
- Common issue text.

The search suggestions are:

- Sony A6400.
- MacBook Air M2.
- Trek Marlin 5.
- Dell Ultrasharp 27.

Search results are displayed as product cards. Each card shows:

- Bookmark button in the top-right corner.
- Category.
- Brand and model.
- Year.
- Recommendation badge.
- Used average price.
- Fair price.
- MSRP drop/depreciation.
- Reliability score.
- Demand score.
- Scam probability score.
- Short recommendation explanation.
- Link to the product insight page.

The bookmark button saves the product guide into the Saved tab with the product's current fair price, "Other" as the source, and a note that it was saved from the price guide. If Supabase is configured and the user is logged in, it writes to `saved_items`; otherwise it saves locally in browser local storage. The same product-card bookmark appears on the homepage's popular buyer guides and on related product cards.

If no products match, the page shows an empty state telling the user to try a broader model name, brand, category, or remove the year filter.

Product Insight Pages

Product pages are available at:

```
/products/[id]
```

Each product page is generated from mock product data. The current app uses static params for all mock product ids.

Each product page includes:

- Back to search link.
- Category pill.
- Year.
- Product brand and model.
- Recommendation explanation.
- Recommendation card.
- Reliability score.
- Demand score.
- Scam probability score.
- Price summary.
- Depreciation chart.
- Common problems.
- Seller questions.
- Scam probability warnings.
- Buying checklist.
- Embedded listing analyzer preloaded with that product.
- Save listing form.
- Best years/models.
- Models to avoid.
- Mock marketplace listings.
- Similar alternatives.

The product insight page is useful when a user wants a reference guide before or while checking a specific listing.

Price Summary

The price summary shows:

- Original MSRP.
- Used average.
- Fair price range.
- Depreciation.

It labels the price summary as:

- Strong used value.
- Normal market price.

- Sellers are asking high.

The label is based on used average compared with fair price.

Depreciation Chart

The depreciation chart uses Recharts and data from:

`lib/depreciation.ts`

It shows estimated value over time with a line chart.

Common Problems

The common problems section shows product-specific inspection concerns with severity labels:

- Low.
- Medium.
- High.

Seller Questions

Each product has category-specific seller questions. These are intended to help the buyer ask about ownership, condition, hidden defects, accessories, and proof before meeting.

Scam Probability Warnings

The scam probability warning section shows a global list of common used-market red flags:

- Price far below normal market value.
- Seller refuses public meetup or local pickup.
- Listing uses stock photos only.
- Seller will not share serial number or proof of ownership.
- Seller adds urgency pressure or says other buyers are waiting.
- Seller asks for payment before pickup.
- Answers are vague, copied, or avoid basic condition questions.

Buying Checklist

Each product has a category-specific buying checklist. The checklist is meant to be used before payment or during a meetup.

Embedded Listing Analyzer

The product page includes the same listing analyzer used on `/submit`, but preloaded with the current product. This means the user can read the product guide and immediately analyze a seller listing for that same model.

Save Listing

The save form lets the user track a listing for that product. It asks for:

- Asking price.
- Marketplace.
- Seller location.
- Notes.

If Supabase is configured and the user is logged in, the saved item is written to the Supabase `saved_items` table. Otherwise, it is saved locally in browser local storage.

Saved Items

The saved items page is available at:

`/saved`

Saved item behavior lives mainly in:

- `components/SavedItemsClient.tsx`
- `components/SavedItemCard.tsx`
- `lib/savedItemsStorage.ts`

Saved items can come from:

- Supabase, if environment variables are configured and the user is logged in.
- Browser local storage, if Supabase is not configured or the user is not logged in.
- Product-card bookmarks from search, homepage buyer guides, and related product cards.
- Analyzer verdict saves from the result card after checking a resale or retail link.

The local storage key is:

`buywise.saved-items`

Each saved item includes:

- Id.
- Optional user id.
- Product id.
- Asking price.

- Marketplace.
- Seller location.
- Notes.
- Status.
- Created date.

Allowed statuses are:

- watching.
- contacted.
- negotiating.
- bought.
- passed.

On the saved items page, each card can show:

- Marketplace.
- Product name.
- Seller location.
- Asking price.
- Fair value.
- Difference from fair value.
- Product year.
- Notes.
- Status dropdown.
- View insight link.
- Delete button.

If the user is not logged in, the page shows a notice explaining that they are viewing browser-saved items. If Supabase is configured, the notice tells them to log in to sync items to their account. If Supabase is not configured, the notice says saved items are stored in the browser for now.

If there are no saved items, the page shows an empty state telling the user to use the bookmark on a product card or save a verdict after analyzing a link.

Auth

The auth page is available at:

```
/auth
```

Auth is backed by Supabase if the following env vars exist:

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`

The Supabase client lives in:

`lib/supabaseClient.ts`

If either env var is missing, `supabase` is set to `null`, and the app falls back to local behavior where possible.

The auth form supports:

- Login.
- Signup.

The form validates:

- Email must include `@`.
- Password must be at least 6 characters.

If Supabase is not configured, the form shows setup instructions and does not attempt login.

On signup, the app upserts a row into the `profiles` table with:

- User id.
- Email.

The auth page is available from the footer Account link and the navbar Log In button.

Admin Page

The admin page is available at:

`/admin`

It is a mock data overview page, not a protected production admin system.

It shows:

- Total product count.
- Total MSRP tracked.
- Total fair market value tracked.
- Category cards.
- Mock product lists by category.

- Integration-ready service file list.
- Customer-facing page note.
- Link to the About Us page.
- Table of all mock products.

The admin page helps inspect the current mock catalog and service architecture.

About Us Page

The customer-facing explanation page is available at:

`/about`

The old `/monetization` route redirects to `/about`.

This page is designed for potential users, not internal planning. It explains what BuyWise does, who it helps, and why a buyer should use it before paying for a used listing or retail bargain.

The main message is:

Know if a used listing or retail deal is worth buying before you pay.

The page explains that users can paste a product link and get:

- Deal verdict.
- Risk level.
- Confidence score.
- Suggested offer or buy-under range.
- Reasons the link might not be worth it.
- Questions to ask the seller.
- Buyer checklist.
- Resale and retail alternatives.

It also explains the four main benefits:

- Start with a link.
- See if the price makes sense.
- Catch risky listings.
- Compare better options.

The page puts credibility stats from the current catalog near the top:

- MSRP tracked.

- Used fair value tracked.
- Categories covered.
- Product guides.

The page is aimed at:

- Students buying laptops or MacBooks.
- Creators buying used cameras.
- Parents buying used gear.
- People checking bikes, monitors, and electronics.
- Anyone who wants a second opinion before paying.

The page is honest that the current preview uses a starter catalog for cameras, laptops, bikes, and monitors, and that link reading works when public pages expose readable product data.

Mock Product Catalog

The current mock catalog lives in:

`data/mockProducts.ts`

The MVP includes 20 products:

Cameras

- Sony A6400, 2019.
- Canon EOS R10, 2022.
- Fujifilm X-T30 II, 2021.
- Sony A7 III, 2018.
- Canon M50 Mark II, 2020.

Laptops

- Apple MacBook Air M1, 2020.
- Apple MacBook Air M2, 2022.
- Dell XPS 13, 2022.
- Lenovo ThinkPad X1 Carbon, 2022.
- Microsoft Surface Laptop 5, 2022.

Bikes

- Trek Marlin 5, 2022.

- Specialized Rockhopper, 2021.
- Giant Talon, 2022.
- Cannondale Quick, 2021.
- Trek FX 3, 2022.

Monitors

- Dell UltraSharp U2720Q, 2020.
- LG 27GL850, 2019.
- ASUS ProArt PA278QV, 2020.
- Samsung Odyssey G5, 2021.
- BenQ PD2700U, 2018.

Each product contains:

- Id.
- Category.
- Brand.
- Model.
- Year.
- Original MSRP.
- Used low.
- Used average.
- Used high.
- Fair price.
- Depreciation percent.
- Reliability score.
- Demand score.
- Scam probability score.
- Common issues.
- Best years/models.
- Models to avoid.
- Buying checklist.
- Seller questions.
- Recommendation.
- Recommendation explanation.

Product ids are used for product routes and saved items.

Data Types

Core types live in:

`types/index.ts`

Main product categories currently supported:

- Cameras.
- Laptops.
- Bikes.
- Monitors.

Recommendation labels:

- Great deal.
- Fair price.
- Overpriced.
- Risky purchase.
- Avoid.

Risk levels:

- Low.
- Medium.
- High.

Saved item statuses:

- watching.
- contacted.
- negotiating.
- bought.
- passed.

Marketplace sources:

- eBay.
- Craigslist.
- Facebook Marketplace.
- OfferUp.
- Local seller.

- Other.

Link analysis modes:

- resale.
- retail.

Risk signal severity:

- low.
- medium.
- high.

The app also defines link-analysis and extraction structures:

- `ListingAlternative`.
- `ListingAnalysisContext`.
- `LinkExtractionResult`.

`ListingAlternative` describes a better resale or retail/MSRP option from the mock catalog. It includes product id, title, price, price label, source type, recommended action, expected outcome, and reason.

`ListingAnalysisContext` describes the analyzed link context. It includes mode, URL, source label, source domain, link price, benchmark label, matched product name, resale alternatives, and retail alternatives.

`LinkExtractionResult` describes what the server was able to read from a pasted link. It includes success/manual flags, URL, source label, source domain, mode, marketplace, title, description, price, product name, confidence, message, and optional warnings.

Supabase Schema

The schema lives in:

`supabase/schema.sql`

Tables currently defined:

- `profiles`.
- `products`.
- `product_issues`.
- `buying_checklist_items`.
- `seller_questions`.
- `saved_items`.
- `listing_checks`.

The app currently reads product data from `data/mockProducts.ts`. The Supabase product tables are scaffolded for a future migration of the mock catalog into the database.

profiles

Stores:

- id.
- email.
- created_at.

The id references `auth.users(id)`.

products

Stores:

- id.
- category.
- brand.
- model.
- year.
- msrp.
- used_low.
- used_avg.
- used_high.
- fair_price.
- depreciation_percent.
- reliability_score.
- demand_score.
- scam_risk_score, currently displayed to users as scam probability.
- recommendation.
- recommendation_explanation.
- created_at.

product_issues

Stores product-specific issues:

- id.

- product_id.
- issue.
- severity.

buying_checklist_items

Stores product-specific checklist items:

- id.
- product_id.
- checklist_item.

seller_questions

Stores product-specific seller questions:

- id.
- product_id.
- question.

saved_items

Stores saved listings:

- id.
- user_id.
- product_id.
- asking_price.
- marketplace.
- seller_location.
- notes.
- status.
- created_at.

Status must be one of:

- watching.
- contacted.
- negotiating.
- bought.
- passed.

listing_checks

Stores analyzer runs when Supabase and auth are available:

- id.
- user_id.
- product_id.
- asking_price.
- condition.
- description.
- marketplace.
- deal_score.
- risk_level.
- confidence_score.
- price_difference.
- red_flags.
- positive_signals.
- suggested_offer_low.
- suggested_offer_high.
- created_at.

The `red_flags` and `positive_signals` fields are JSONB arrays.

Row Level Security

RLS is enabled for:

- profiles.
- saved_items.
- listing_checks.

Policies:

- Profiles are readable by owner.
- Profiles are insertable by owner.
- Saved items are owned by user.
- Listing checks are owned by user, with support for nullable user_id.

Marketplace Services

Mock marketplace service files:

- `services/ebayService.ts`.
- `services/craigslistService.ts`.
- `services/facebookMarketplaceService.ts`.
- `services/priceAnalysisService.ts`.

These services return typed `MarketplaceListing` objects. They are async so they can be replaced later with real API calls.

The `getMarketplaceSignals(product)` function calls:

- `getEbayCompletedListings(product)`.
- `getCraigslistLocalListings(product)`.
- `getFacebookMarketplaceListings(product)`.

It combines and sorts listings by price.

The app does not scrape Facebook Marketplace. Facebook Marketplace data is mocked.

Future real marketplace integration should:

1. Keep the same return type.
2. Use approved APIs or user-submitted listing data.
3. Store provider credentials in server-only environment variables.
4. Move provider calls into server routes or server actions.
5. Normalize results into `MarketplaceListing`.
6. Avoid scraping Facebook Marketplace.

Current UI and Design Direction

The current app is designed to feel like a clean consumer buyer tool.

Important visual direction:

- Simple and fast.
- Mobile-friendly.
- Practical and trustworthy.
- Marketplace/tool-like.
- Buyer-focused.

Current style details:

- Tailwind theme uses `ink`, `paper`, `mint`, `amber`, and `danger`.
- Cards use small border radius.
- UI uses lucide-react icons.
- Buttons and inputs use a shared `focus-ring` class.
- The app avoids a heavy finance-dashboard feel.
- The homepage avoids generic marketing-first layout and puts the link checker action first.

The main user will often be on mobile while browsing a marketplace or retail app. Mobile behavior matters because a user may paste a link quickly, check the verdict, copy seller questions, compare alternatives, and use the checklist at a meetup or before checkout.

What Is Implemented Right Now

The current version can:

- Show a polished homepage focused on dropping resale or retail links.
- Accept a quick link draft on the homepage.
- Try to auto-populate homepage fields from safe public product/listing links.
- Move that draft into the full analyzer.
- Try to auto-populate analyzer fields from safe public product/listing links.
- Extract readable page metadata, structured product names, descriptions, and prices when the source exposes them.
- Safely reject local/private links and Facebook Marketplace links.
- Infer link source labels from common resale and retail domains.
- Infer whether a link is likely resale or retail.
- Analyze a resale listing or retail product link with local deterministic logic.
- Compare resale link price to mock fair-market values.
- Compare retail link price to mock MSRP benchmarks.
- Detect red flag wording in seller/listing text.
- Detect positive trust signals in seller/listing text.
- Adjust risk based on marketplace source.
- Adjust score based on condition.
- Return a recommendation label.
- Return a deal score.
- Return a confidence score.
- Return a risk level.
- Return a benchmark gap.
- Return a suggested offer range or target buy range.

- Return a negotiation tip.
- Return next steps.
- Show a visible "Why this might NOT be worth it" section.
- Show better resale moves when the mock catalog has them.
- Show better retail moves when the mock catalog has them.
- Explain what action to take for each alternative.
- Show product-specific seller questions.
- Let users copy seller questions from the result.
- Show product-specific meetup checklist items.
- Search the mock product catalog.
- Filter products by category and minimum year.
- View product insight pages.
- View price summaries.
- View depreciation charts.
- View common issues.
- View scam probability warnings.
- View buying checklists.
- Bookmark product cards from search, homepage guides, and related product cards.
- Save analyzed link verdicts from the result card.
- Save product/listing details locally.
- Sync saved items to Supabase if configured and logged in.
- Update saved item status.
- Delete saved items.
- Sign up and log in with Supabase if configured.
- Save listing check results to Supabase if configured and logged in.
- Show admin/mock data overview.
- Show a customer-facing About Us page.

What Is Not Implemented Yet

The current version does not:

- Use OpenAI API analysis.
- Have an `app/api/analyze-listing/route.ts` route.
- Guarantee live page extraction works for every pasted link.
- Extract images, condition, shipping, seller profiles, or private marketplace details from URLs.

- Use computer vision.
- Upload screenshots.
- Extract text from listing screenshots.
- Analyze product photos.
- Scrape Facebook Marketplace.
- Call real eBay/Craigslist/Facebook/OfferUp APIs.
- Call real retail APIs.
- Seed Supabase products from the mock catalog automatically.
- Require accounts for local use.
- Process payments.
- Send price alerts.
- Generate paid reports.
- Provide seller verification.
- Have a browser extension.
- Have a mobile app.
- Include cars.
- Include phones, game consoles, tools, furniture, or sneakers in the current mock catalog.

Phones, game consoles, and tools are desired future categories. Cars are intentionally excluded for now because car purchases involve VINs, titles, accident history, financing, insurance, local laws, and higher liability.

Current Limitations

The largest current limitations are:

- Market prices are mock estimates.
- Product catalog only covers 20 products.
- Listing checks are calculated on the client.
- The analyzer uses keyword/rule-based logic, not a language model.
- Product matching is simple string matching against brand and model.
- Link extraction depends on what the public page exposes in HTML metadata.
- Some retailers and marketplaces block automated reads or render details only in client-side scripts.
- Extracted prices may need user review because pages can contain multiple dollar amounts.
- Pasted URLs are not used to scrape Facebook Marketplace.
- Retail alternatives are mock MSRP benchmarks, not real store availability.
- No image/screenshot support.
- No real marketplace integrations.

- Supabase is optional and may not be configured.
- Saved items are local to the browser when not logged in.
- Listing check history is only saved to Supabase when logged in.
- Product pages use mock marketplace listings.

These limitations are acceptable for the current MVP because the goal is to make the core buyer workflow feel useful before adding API cost and complexity.

Future-Ready Architecture

The current structure intentionally leaves room for OpenAI analysis later.

It now includes a basic server-side link extraction route that can be expanded later. The current route should eventually be backed by provider-specific parsers, approved APIs, or affiliate/product feeds for better accuracy.

Recommended future link analysis improvements:

- Keep URL/source inference in a shared helper.
- Fetch only from allowed providers or approved APIs.
- Do not scrape Facebook Marketplace.
- Normalize extracted page data into a structured listing/product object.
- Add provider-specific extraction for eBay, Craigslist, major retailers, and approved product APIs.
- Add stronger price disambiguation when pages show sale price, MSRP, shipping, coupons, and financing.
- Continue using `lib/dealQuality.ts` for deterministic pricing/risk logic.
- Return graceful fallback instructions when a link cannot be fetched.

Recommended future OpenAI architecture:

- Add server route: `app/api/analyze-listing/route.ts`.
- Add service: `services/openaiListingAnalysisService.ts`.
- Keep deterministic price/deal logic in `lib/dealQuality.ts`.
- Let OpenAI extract structured listing facts and wording risks.
- Do not let OpenAI become the only source of truth for pricing.
- Fall back to local analyzer if the API key is missing.

Potential OpenAI use cases later:

- Extract title, brand, model, condition, and price from pasted text.
- Detect scam language beyond exact keyword matches.
- Generate better seller questions.
- Explain verdicts in plainer language.

- Summarize seller replies.
- Identify missing proof signals.

Screenshot/photo support can come after text analysis. Future image support could:

- Let users upload listing screenshots.
- Extract title, price, description, location, and seller wording.
- Detect stock images.
- Detect blurry photos.
- Check for missing serial number photos.
- Look for visible damage.
- Look for missing accessories.

Suggested Future Product Order

Based on the current direction, the best next build order is:

1. Improve provider-specific link extraction accuracy.
2. Add approved product/price APIs or affiliate feeds for real alternatives.
3. Add OpenAI-powered text analysis only after link extraction and UX feel good.
4. Add better saved-deal workflows.
5. Add phones, game consoles, and tools.
6. Add screenshot/listing image upload.
7. Add real sold-price data or approved APIs.
8. Add premium reports or alerts.
9. Add seller verification only when trust rules are stronger.

Do not add cars yet.

Do not scrape Facebook Marketplace.

Do not monetize in a way that makes recommendations feel biased.

How A User Should Experience BuyWise Today

The ideal current user flow is:

1. User finds a resale listing or retail product page.
2. User opens BuyWise.
3. User drops the link into the homepage.

4. BuyWise sends them to the full analyzer step when they click Analyze link.
5. BuyWise tries to auto-fill title, price, description, source, mode, and closest mock price guide.
6. User reviews or corrects the auto-filled details and adds anything missing.
7. User checks or adjusts the closest mock price guide.
8. User clicks Analyze link.
9. BuyWise returns a buyer verdict.
10. User reads the offer or target buy range, "Why this might NOT be worth it," red flags, trust signals, alternatives, and next steps.
11. User compares better resale moves and better retail moves if any are shown.
12. User copies seller questions or uses the checklist before checkout/meeting.
13. User can save the item and track status if they want.

The product should help users avoid bad buys before they waste time messaging, driving, checking out, or paying.

Maintenance Note

This overview is intended to be a living project document.

When the website behavior changes, update this overview and regenerate `BuyWise Overview.pdf`. Future updates should keep the document focused on what the current version actually does, clearly separating implemented features from planned features.

The current PDF is generated from this Markdown source:

`BuyWise Overview.md`

The current PDF output is:

`BuyWise Overview.pdf`